

Random–Node RRT

Probabilistic Completeness & Exploration Efficiency

Probabilistic Completeness (sketch)

1. **Positive–probability sampling.**

Every free configuration $q \in \mathcal{C}_{\text{free}}$ lies in the finite workspace rectangle. The algorithm draws samples uniformly inside that rectangle and rejects only those in collision, so the probability of sampling any open neighbourhood $\mathcal{N}_\varepsilon(q)$ is strictly > 0 .

2. **Positive–probability expansion.**

Each iteration chooses a tree node v *uniformly at random*. Hence every existing node has a fixed, non–zero chance of being selected on each trial.

3. **ε –net along a feasible path.**

If a collision–free path τ exists from the (random) start c_0 to the (random) goal g^* , cover τ with finitely many balls of radius $0 < \varepsilon < \text{step}$:

$$c_0 = \tau(t_0), \tau(t_1), \dots, \tau(t_k) = g^*.$$

Whenever (a) the sample falls inside the next ball and (b) the randomly chosen node is the previous waypoint, one **steer** hop adds $\tau(t_i)$ to the tree in a single, collision–free step.

4. **Infinite–trial guarantee.**

Because steps 1–2 give every missing waypoint a constant, independent chance of being added at each iteration, the probability that the whole ε –net is *never* completed tends to 0 as the iteration count $\rightarrow \infty$.

$$\implies \Pr(\text{algorithm finds a path} \mid \text{path exists}) \xrightarrow[N \rightarrow \infty]{} 1.$$

Hence the algorithm is *probabilistically complete*.

Exploration Efficiency

Criterion	Random-node variant	Classical “nearest-node” RRT
Frontier bias	None — the node to expand is chosen uniformly from the entire tree.	Strong — Voronoi bias naturally steers growth toward unexplored regions.
Node utility	Quickly re-selects interior nodes $\rightarrow$ many wasted extensions.	Newly added samples almost always lie near the frontier.
Scalability	Performance degrades after a few thousand nodes.	Scales well even with tens of thousands of nodes when a k -d tree is used for nearest-neighbour queries.
Implementation cost	Very small (no nearest-neighbour data structure required).	Needs a k -d tree or grid for fast nearest search.

Verdict. The planner does cover configuration space eventually, but it does so *inefficiently*: most iterations re-explore already-filled regions, so convergence is much slower than standard RRT or RRT*.